

Name :- Varsharani Pavankumar Kasliwal

Roll No. :- CE14

Aim

To implement Support Vector Machine (SVM) for classification of the Iris dataset and visualize its decision boundary and compare it with a normal linear classification boundary.

Objectives

1. To understand the concept of Support Vector Machine.
2. To implement SVM using Python.
3. To visualize the SVM decision boundary.
4. To identify the support vectors.
5. To compare the SVM boundary with a normal linear classification boundary.
6. To evaluate the performance of the classifiers.

Relevant Theory

Support Vector Machine

Support Vector Machine (SVM) is a supervised machine learning algorithm used for classification.

SVM finds a decision boundary that separates different classes while trying to maximize the distance, called the **margin**, between the classes.

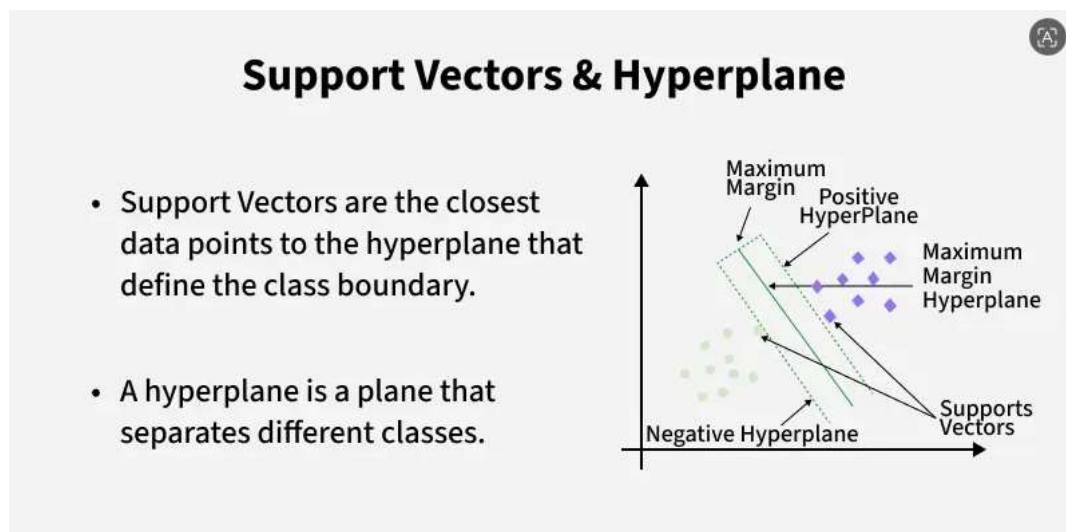
The points closest to the decision boundary are called **support vectors**.

For a linear SVM, the decision boundary can be represented as:

$$w^T x + b = 0$$

where:

- w = weight vector
- x = input features
- b = bias



Kernel

SVM can use different kernels to create decision boundaries.

Common kernels are:

- Linear
- Polynomial
- RBF
- Sigmoid

In this experiment, we mainly use the **Linear SVM** and **RBF SVM**.

Dataset Information

Get the IRIS dataset from Kaggle using below link

<https://www.kaggle.com/datasets/uciml/iris>

The Iris dataset contains measurements of Iris flowers belonging to three classes:

- Iris-setosa
- Iris-versicolor
- Iris-virginica

The four features are:

- Sepal Length
- Sepal Width
- Petal Length
- Petal Width

For visualization, we will use:

Petal Length and Petal Width.

Task 1: Import Libraries and Load Dataset

Load the Iris dataset.

```
import pandas as pd
from sklearn.datasets import load_iris

# Load Iris dataset
iris = load_iris()

# Create DataFrame
df = pd.DataFrame(iris.data, columns=iris.feature_names)

# Add target column
df['target'] = iris.target

# Display the entire DataFrame
print(df)
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	\
0	5.1	3.5	1.4	0.2	
1	4.9	3.0	1.4	0.2	
2	4.7	3.2	1.3	0.2	
3	4.6	3.1	1.5	0.2	
4	5.0	3.6	1.4	0.2	
..	...	...	...	...	
145	6.7	3.0	5.2	2.3	
146	6.3	2.5	5.0	1.9	
147	6.5	3.0	5.2	2.0	
148	6.2	3.4	5.4	2.3	
149	5.9	3.0	5.1	1.8	

	target
0	0
1	0
2	0
3	0
4	0
..	...
145	2
146	2
147	2
148	2
149	2

[150 rows x 5 columns]

Task 2: Explore the Dataset

```
# Display the shape of the DataFrame
print("Shape of the DataFrame:", df.shape)

# Display data types of each column
```

```
print("\nData types of columns:")
print(df.info())

# Display descriptive statistics
print("\nDescriptive statistics:")
print(df.describe())

# Check for missing values
print("\nMissing values per column:")
print(df.isnull().sum())
```

Shape of the DataFrame: (150, 5)

```
Data types of columns:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
#   Column                Non-Null Count  Dtype
---  -
0   sepal length (cm)      150 non-null   float64
1   sepal width (cm)       150 non-null   float64
2   petal length (cm)      150 non-null   float64
3   petal width (cm)       150 non-null   float64
4   target                 150 non-null   int64
dtypes: float64(4), int64(1)
memory usage: 6.0 KB
None
```

```
Descriptive statistics:
      sepal length (cm)  sepal width (cm)  petal length (cm)  \
count      150.000000      150.000000      150.000000
mean         5.843333         3.057333         3.758000
std          0.828066         0.435866         1.765298
min          4.300000         2.000000         1.000000
25%          5.100000         2.800000         1.600000
50%          5.800000         3.000000         4.350000
75%          6.400000         3.300000         5.100000
max          7.900000         4.400000         6.900000
```

```
      petal width (cm)  target
count      150.000000  150.000000
mean         1.199333     1.000000
std          0.762238     0.819232
min          0.100000     0.000000
25%          0.300000     0.000000
50%          1.300000     1.000000
75%          1.800000     2.000000
max          2.500000     2.000000
```

```
Missing values per column:
sepal length (cm)    0
sepal width (cm)     0
petal length (cm)    0
petal width (cm)     0
target               0
dtype: int64
```

Task 3: Select Features and Target

Remove the **Id** column.

Select Remaining Features

```
# Select features (X) and target (y)
X = df.drop('target', axis=1)
y = df['target']

print("Features (X) shape:", X.shape)
print("Target (y) shape:", y.shape)
```

```
Features (X) shape: (150, 4)
Target (y) shape: (150,)
```

Task 4: Split and Scale the Data

Divide the dataset into training and testing sets.

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Split the dataset into training and testing sets
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)

# Scale the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("\nX_train_scaled (first 5 rows):\n", X_train_scaled[:5])
print("\nX_test_scaled (first 5 rows):\n", X_test_scaled[:5])
```

```
X_train shape: (105, 4)
X_test shape: (45, 4)
y_train shape: (105,)
y_test shape: (45,)

X_train_scaled (first 5 rows):
[[-0.4134164 -1.46200287 -0.09951105 -0.32339776]
 [ 0.55122187 -0.50256349  0.71770262  0.35303182]
 [ 0.67180165  0.21701605  0.95119225  0.75888956]
 [ 0.91296121 -0.02284379  0.30909579  0.2177459 ]
 [ 1.63643991  1.41631528  1.30142668  1.70589097]]

X_test_scaled (first 5 rows):
[[ 0.3100623 -0.50256349  0.484213 -0.05282593]
 [-0.17225683  1.89603497 -1.26695916 -1.27039917]
 [ 2.23933883 -0.98228318  1.76840592  1.43531914]
 [ 0.18948252 -0.26270364  0.36746819  0.35303182]
 [ 1.15412078 -0.50256349  0.54258541  0.2177459 ]]
```

Task 5: Implement Linear SVM

Create an SVM model using a **linear kernel**.

Make predictions.

```
from sklearn.svm import SVC

# Create an SVM model with a linear kernel
svm_linear_model = SVC(kernel='linear', random_state=42)

# Train the model
svm_linear_model.fit(X_train_scaled, y_train)

# Make predictions on the scaled test data
y_pred_linear_svm = svm_linear_model.predict(X_test_scaled)

print("First 10 predictions of Linear SVM:", y_pred_linear_svm[:10])
print("First 10 actual values:", y_test.values[:10])

First 10 predictions of Linear SVM: [1 0 2 1 1 0 1 2 2 1]
First 10 actual values: [1 0 2 1 1 0 1 2 1 1]
```

Task 6: Evaluate SVM

Calculate Accuracy

Calculate the accuracy of the SVM model.

Confusion Matrix

Generate the confusion matrix to evaluate the classification results.

Classification Report

Generate the classification report containing:

- Precision
- Recall
- F1-Score
- Support

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

```
# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred_linear_svm)
print(f"Accuracy of Linear SVM: {accuracy:.4f}")

# Generate confusion matrix
conf_matrix = confusion_matrix(y_test, y_pred_linear_svm)
print("\nConfusion Matrix for Linear SVM:\n", conf_matrix)

# Generate classification report
class_report = classification_report(y_test, y_pred_linear_svm)
print("\nClassification Report for Linear SVM:\n", class_report)
```

Accuracy of Linear SVM: 0.9778

Confusion Matrix for Linear SVM:

```
[[19  0  0]
 [ 0 12  1]
 [ 0  0 13]]
```

Classification Report for Linear SVM:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	19
1	1.00	0.92	0.96	13
2	0.93	1.00	0.96	13
accuracy			0.98	45
macro avg	0.98	0.97	0.97	45
weighted avg	0.98	0.98	0.98	45

Task 7: Visualize SVM Decision Boundary

For visualization, we use only two features:

- Petal Length
- Petal Width

Create the 2-D Dataset

Select **Petal Length** and **Petal Width** as the input features.

Split the Data

Divide the 2-D dataset into training and testing sets.

Scale the Data

Standardize the selected features before training the SVM model.

Train SVM

Train an SVM classifier using a **linear kernel**.

Plot Decision Boundary

Visualize the SVM decision boundary along with the data points.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

# Create the 2-D Dataset using Petal Length and Petal Width
X_2d = df[['petal length (cm)', 'petal width (cm)']]
y_2d = df['target']

# Split the 2-D data
X_2d_train, X_2d_test, y_2d_train, y_2d_test = train_test_split(X_2d, y_2d, test_size=0.3, random_state=42)

# Scale the 2-D data
scaler_2d = StandardScaler()
X_2d_train_scaled = scaler_2d.fit_transform(X_2d_train)
X_2d_test_scaled = scaler_2d.transform(X_2d_test)

# Train SVM with linear kernel on 2-D data
svm_linear_2d = SVC(kernel='linear', random_state=42)
svm_linear_2d.fit(X_2d_train_scaled, y_2d_train)
```

```

# Function to plot decision boundary
def plot_decision_boundary(X_scaled, y, model, title):
    plt.figure(figsize=(10, 7))
    # Create a mesh to plot in
    x_min, x_max = X_scaled[:, 0].min() - 1, X_scaled[:, 0].max() + 1
    y_min, y_max = X_scaled[:, 1].min() - 1, X_scaled[:, 1].max() + 1
    xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.02),
                          np.arange(y_min, y_max, 0.02))

    Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)
    plt.contourf(xx, yy, Z, alpha=0.8, cmap=plt.cm.RdYlBu)

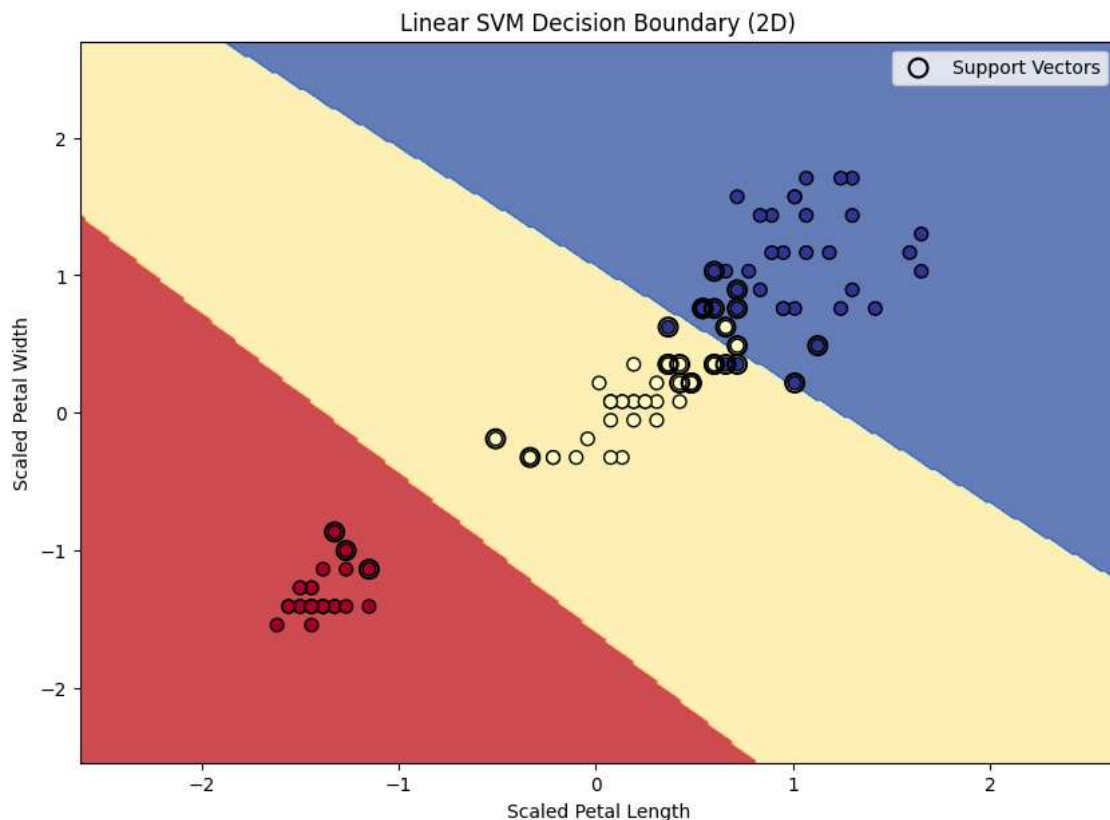
    # Plot data points
    scatter = plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=y, cmap=plt.cm.RdYlBu, edgecolors='k', s=50)

    # Plot support vectors if available (for SVC)
    if hasattr(model, 'support_vectors_'):
        plt.scatter(model.support_vectors_[:, 0],
                    model.support_vectors_[:, 1],
                    s=100, facecolors='none', edgecolors='k', linewidths=1.5, label='Support Vectors')

    plt.xlabel('Scaled Petal Length')
    plt.ylabel('Scaled Petal Width')
    plt.title(title)
    plt.legend(*scatter.legend_elements(), title="Classes")
    if hasattr(model, 'support_vectors_'):
        plt.legend()
    plt.show()

# Plot the decision boundary for Linear SVM on 2-D data
plot_decision_boundary(X_2d_train_scaled, y_2d_train, svm_linear_2d, 'Linear SVM Decision Boundary (2D)')

```



Task 8: Normal Linear Classification Boundary

For comparison, we use **Logistic Regression** as a normal linear classifier.

Train a Logistic Regression model using the same two features and visualize its decision boundary.

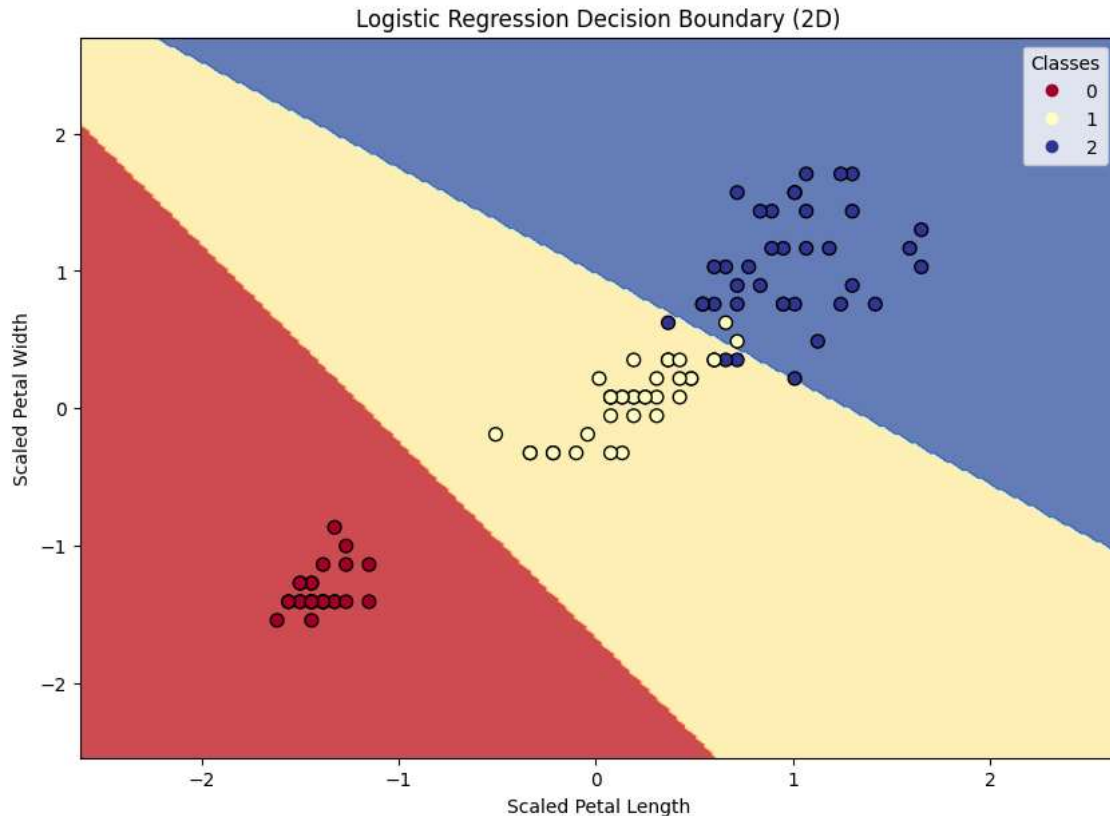
```

from sklearn.linear_model import LogisticRegression

# Train Logistic Regression model on 2-D data
logistic_reg_2d = LogisticRegression(random_state=42)
logistic_reg_2d.fit(X_2d_train_scaled, y_2d_train)

```

```
# Plot the decision boundary for Logistic Regression on 2-D data
plot_decision_boundary(X_2d_train_scaled, y_2d_train, logistic_reg_2d, 'Logistic Regression Decision Boundary (2D)')
```



Task 9: Compare SVM and Normal Boundary

Plot both the **SVM** and **Logistic Regression** decision boundaries on the same graph.

Compare how the two classifiers separate the different classes.

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.lines import Line2D

plt.figure(figsize=(10, 7))

# Create a mesh to plot in (re-using variables from Task 7)
x_min, x_max = X_2d_train_scaled[:, 0].min() - 1, X_2d_train_scaled[:, 0].max() + 1
y_min, y_max = X_2d_train_scaled[:, 1].min() - 1, X_2d_train_scaled[:, 1].max() + 1
xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.02),
                     np.arange(y_min, y_max, 0.02))

# Plot data points
scatter = plt.scatter(X_2d_train_scaled[:, 0], X_2d_train_scaled[:, 1], c=y_2d_train,
                    cmap=plt.cm.RdYlBu, edgecolors='k', s=50, zorder=2)

# Plot SVM Decision Boundary Lines
Z_svm = svm_linear_2d.predict(np.c_[xx.ravel(), yy.ravel()])
Z_svm = Z_svm.reshape(xx.shape)
plt.contour(xx, yy, Z_svm, colors='blue', levels=np.arange(svm_linear_2d.classes_.min() + 0.5, svm_linear_2d.classes_.max(), 1),
            linestyle='--', linewidths=2, zorder=1)

# Plot Logistic Regression Decision Boundary Lines
Z_logreg = logistic_reg_2d.predict(np.c_[xx.ravel(), yy.ravel()])
Z_logreg = Z_logreg.reshape(xx.shape)
plt.contour(xx, yy, Z_logreg, colors='green', levels=np.arange(logistic_reg_2d.classes_.min() + 0.5, logistic_reg_2d.classes_.max(), 1),
            linestyle='--', linewidths=2, zorder=1)

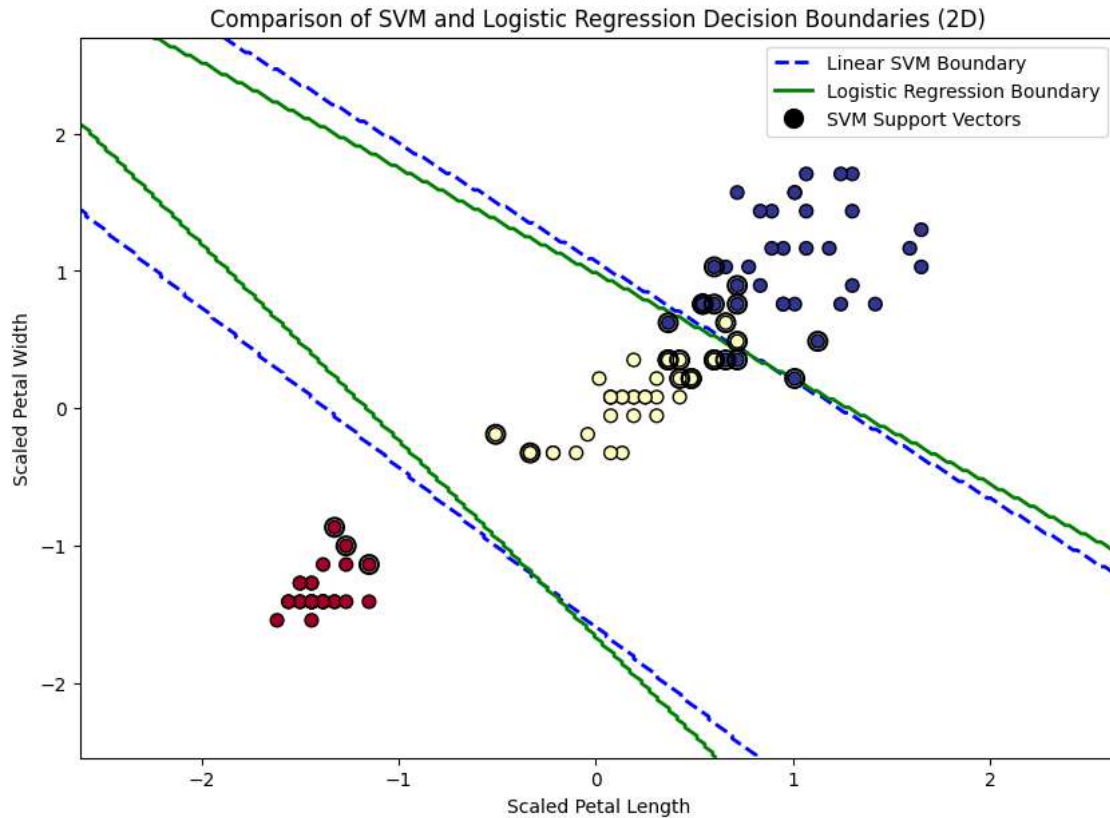
# Plot SVM Support Vectors
plt.scatter(svm_linear_2d.support_vectors_[:, 0],
            svm_linear_2d.support_vectors_[:, 1],
            s=100, facecolors='none', edgecolors='black', linewidths=1.5, label='SVM Support Vectors', zorder=3)

plt.xlabel('Scaled Petal Length')
plt.ylabel('Scaled Petal Width')
```

```
plt.title('Comparison of SVM and Logistic Regression Decision Boundaries (2D)')

# Custom legend handles
custom_lines = [
    Line2D([0], [0], color='blue', lw=2, linestyle='--', label='Linear SVM Boundary'),
    Line2D([0], [0], color='green', lw=2, linestyle='-', label='Logistic Regression Boundary'),
    Line2D([0], [0], marker='o', color='w', markerfacecolor='black', markeredgcolor='black', markersize=10, lw=0, label='SVM Support Vectors')
]

plt.legend(handles=custom_lines)
plt.show()
```



Task 10: Compare Performance

Compare the performance of the **Linear SVM** and **Logistic Regression** using:

- Accuracy
- Confusion Matrix
- Classification Report

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Make predictions with Logistic Regression on the scaled test data
y_pred_logistic_reg = logistic_reg_2d.predict(X_2d_test_scaled)

# Calculate accuracy for Logistic Regression
accuracy_logreg = accuracy_score(y_2d_test, y_pred_logistic_reg)
print(f"Accuracy of Logistic Regression: {accuracy_logreg:.4f}")

# Generate confusion matrix for Logistic Regression
conf_matrix_logreg = confusion_matrix(y_2d_test, y_pred_logistic_reg)
print("\nConfusion Matrix for Logistic Regression:\n", conf_matrix_logreg)

# Generate classification report for Logistic Regression
class_report_logreg = classification_report(y_2d_test, y_pred_logistic_reg)
print("\nClassification Report for Logistic Regression:\n", class_report_logreg)

print("\n--- Comparison with Linear SVM (from Task 6) ---")
print(f"Linear SVM Accuracy: {accuracy:.4f}")
print("Linear SVM Confusion Matrix:\n", conf_matrix)
print("Linear SVM Classification Report:\n", class_report)
```

Accuracy of Logistic Regression: 1.0000

Confusion Matrix for Logistic Regression:

```
[[19  0  0]
 [ 0 13  0]
 [ 0  0 13]]
```

Classification Report for Logistic Regression:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	19
1	1.00	1.00	1.00	13
2	1.00	1.00	1.00	13
accuracy			1.00	45
macro avg	1.00	1.00	1.00	45
weighted avg	1.00	1.00	1.00	45

--- Comparison with Linear SVM (from Task 6) ---

Linear SVM Accuracy: 0.9778

Linear SVM Confusion Matrix:

```
[[19  0  0]
 [ 0 12  1]
 [ 0  0 13]]
```

Linear SVM Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	19
1	1.00	0.92	0.96	13
2	0.93	1.00	0.96	13
accuracy			0.98	45
macro avg	0.98	0.97	0.97	45
weighted avg	0.98	0.98	0.98	45

Conclusion

The experiment successfully demonstrated the implementation of **Support Vector Machine (SVM)** using the Iris dataset. The dataset was preprocessed and standardized before training the model.

A **Linear SVM** was implemented and its decision boundary was visualized along with the support vectors. A normal linear classification boundary using **Logistic Regression** was also visualized for comparison.